STAGE 4 REPORT

MVP development and execution

Source repository: https://github.com/MathieuMjr/Portfolio_project

INTRODUCTION

The following delivery reports the development cycle across the MVP development of my end-of-the-year personal project at Holberton School, fundamentals year.
For more details on the project itself, please refer to the stage 3 deliverable.

On the basis of the timeline validated in stage 2, I followed a sequential, phase-based approach – closer to a Vee cycle than agile sprints – with development phases for each great part of the architecture: database, back-end and front-end.

Since the delivery assignment asks to design agile sprints, I will detail an agile cycle I could have done in the "retrospective" part.

SPRINT PLANNING / FORECAST TIMELINE (PHASE 3 REMINDER)

The forecast timeline was designed by adapting the steps followed during the Holberton School HBNB project.
Since the development followed a Vee cycle approach, steps were not built according to user stories and sprint goals specifying the delivered functionality at each step.
Gantt diagram is available in Appendix, page 14.
The steps for the MVP development were :

5th – 19th January (2 weeks): main screens prefiguration in HTML5 and CSS3, with static data.
This step helps to identify the content that will be dynamically rendered on the basis of the information sent by the back-end.
Since CSS is a very small part of Holberton School teachings, 2 weeks were intended to this part in order to experiment and learn CSS properly.

19th January – 4th February (2,5 weeks): object creation, app creation, API.

Main objects of the program in Python, facade pattern creation, Flask app initialization and first routes creation.

5th – 23rd February (2,5 weeks): implementing authentication with tokens, password hashing and transition to ORM.

23rd February – 6th March (2 weeks): connection between front-end and back-end via the API. The website is dynamically rendered on the basis of the information received from the back-end and database.

SPRINT REVIEWS WITH TESTS EVIDENCES

This part is the actual timeline of the development. Since a Vee cycle approach has been followed, deliveries are assessed against the forecast timeline.

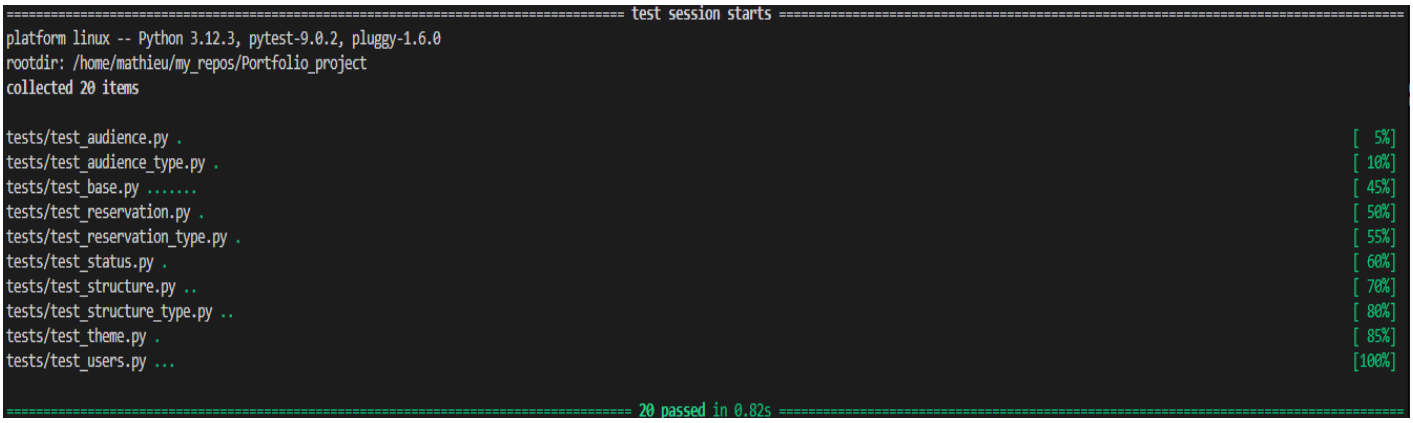
The MVP development started with a 2 week delay due to some essential teaching projects between the 5th of January and the 23rd of January, such as "NodeJS Basics", "Docker", "Typescript" and "Networking basics".
During this phase, work was entirely dedicated to learning those fundamentals.

Consequent adaptations have been made : objects were coded directly as ORM models with functional repositories instead of "in memory" repository.

22nd January - 1st February (10 days) : **Static front-end**.
Main screens (login, planning, reservation) are created with HTML5 and CSS3.
The content is fully static, so it helps to prefigure the dynamic rendering later.
Some time was needed to get familiar with flexbox behavior and responsive units.
This part followed the forecast planning but in a shorter, yet sufficient, amount of time.

2nd - 9th February (7 days) : **Models and persistence**.
Object models and database repositories have been created, directly as models with SQLAlchemy.
The soft delete policy has been defined during the repository methods work and tests, even though the biggest part of the soft delete policy will be in services.
Pydantic validators have been implemented.

Some work has been done to have many configurations such as debug and test configurations.
Tests have been done to ensure objects are properly created, pass the pydantic validators and are persisted, and if relationships are properly implemented.



Tests check if the base class fields properly work and the basic repository methods (soft delete, get all, get_id, update) in the test_base file, and other objects instantiation, persistence, retrieval, etc. in other files.
Relationships, unique constraints, missing fields, bad type or soft deleted object update or retrieval (should work outside of services) are tested too.

For more details on tests:

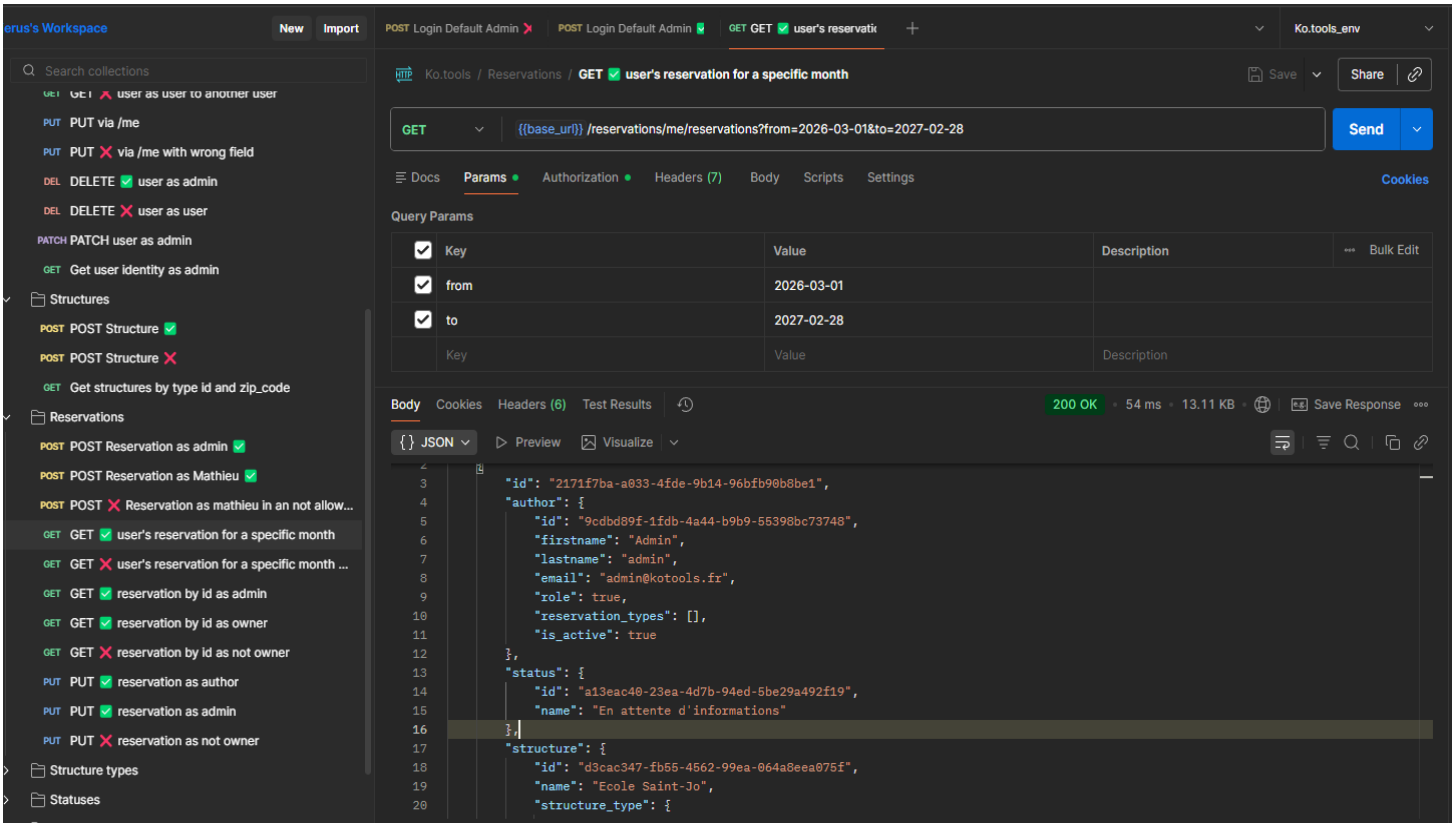
- conftest file with fixtures
- pydantic validators directory
- tests directory

At the end of the sprint, each object is created and persisted correctly with the right relationships. In the forecast timeline, objects were not ORM models at this second step. Going directly into ORM models helped a lot in catching up on the initial delay with a functional delivery.

10th - 19th February (9 days) : **API**.
Bcrypt and Flask JWT have been implemented.
A .env file has been created for sensitive keys.
Routes have been created.
The main objects such as "User", "Reservation" have services because all CRUD operations are covered.

"Structure" object was intended to have all CRUD operations implemented, but only the creation and get operations are implemented – that's why "Structure" has a service.
Catalog objects (reservation types, audience types, statuses...) have a simple GET method.
A seed has been created to populate the database with pre-made catalog objects and a JSON file is generated, ready to import objects name and id in Postman as environment variables.
Tests are executed all along the development phase with Postman to verify and debug body payloads, JSON responses, status codes and errors on each route.

A total of 34 requests tests all the existing routes with correct and bad requests : missing fields/wrong types in the JSON body and good or wrong user token to test authorizations.
Due to time limits, I followed an exploratory manual test approach ; next time automated tests with assertions would be a great improvement.



For more details :

– [Postman collection in JSON format]
– [Environment variable of the Postman collection in JSON format]

This "sprint" delivered a functional back-end. Since there are a lot of objects, a simple seed allowed me to create few catalog objects that were not intended to be manageable in the

MVP, but are needed to create a reservation, making good use of time and helped to catch up on a good part of the initial delay of two weeks.
The structures update is not an available feature ; even though stage three user's stories did not specify clearly it was expected, it was intended to be implemented at this part.

20th February - 2nd March (12 days) : **Client-side rendering.**
JavaScript files are made to dynamically populate some parts of the website content.
On the planning page, user's reservations are displayed as cards for the current month.
Arrows allow the user to navigate through previous and next months.
A "+" icon allows to access the reservation creation screen. The script fetches all the catalog resources to render list boxes (status, reservation type, themes, structure type and structures) or render audience fields of the form.
By clicking on a reservation card, the same reservation page is accessed, but dynamically populated with the data of the wanted reservation. Not updatable fields are disabled, but other fields modifications allow to update the reservation.
Two or three new routes have been necessary.
The code is tested all along the development phase by using the website directly.

This last development part happened as intended in the forecast timeline, delivering a functional and dynamic front-end.
Users can now fully create a reservation, retrieve it and update it, getting an overview of the month's reservations and keeping key information saved, helping them to manage their work efficiently.

Despite the delay, a functional MVP is delivered.
Among the user's stories defined in stage 3, only one prioritized manager user story is not implemented due to the lack of time : "As a manager, I want to create and manage users so my employees can log in to the application and create their reservations".
The intended MySQL database has not been implemented and a SQLite database is used, giving a simpler yet functional alternative considering the initial delay.

RETROSPECTIVE :

What went well:

A MVP is delivered with key features and no known bugs ; despite a 2 weeks delay, the planning was well enough re-ogarnised.

The preparatory design on the ER diagram was good and no design mistakes were discovered during the development : once models were settled, I just had to focus on business logic and soft delete policies without bad surprises.

Back-end responsibilities are pretty well distributed : a dumb repository makes requests, services verify business rules and apply business logic, API receives requests and returned results.

The front-end design is humble but functional, running the core functionalities of the back-end.

What did not go well:

Several issues were revealed while working on the front-end : routes were missing to retrieve user identity (/<user_id> route was dedicated to admin management) and audience types model lacked information to be properly displayed in front, leading to unexpected new back-end developments. The audience types model rebuilt led to bugs and a consequent amount of time dedicated to fixing them.

With no framework for front-end, development has been tedious and exhausting, leading to 300+ lines of code on some pages with big difficulties to maintain and debug. I had to allocate time to refactor the file and functions midway, risking to lose some functional part and time in debugging.

Date formats have been the source of many bugs and long fixes.

What to improve:

In the preparatory part of the project, identifying unfamiliar data types such as dates, and spotting key methods on the different technologies would have helped anticipate the way date data will travel in the application, instead of deciding on a date format in query

parameters that is tricky to generate from client-side.

A better front-end familiarity would have helped to anticipate the fields needed in models to render them on front. Next time, a deeper reflection on the data required for a proper rendering would avoid rebuilding object at the very last moment.

A front-end framework would be a must-have to keep code less tedious to write in rendering functions and easier to read and maintain.

On this Vee approach, a finer definition of final products could have helped to focus on functionality to deliver and a better use of the allocated development time.
For example, Manager authorizations, implemented during the back-end development phase, had no continuity in front-end and would have needed a lot more time to do so.

With this experience, an Agile approach could have produced a better time optimization by putting fully functional features from back-end to front-end at the core of the development process, developing perfectly calibrated features instead of a strong back-end and a lighter front-end.

The following sprint plan illustrates how this MVP could have been structured with an agile approach.

- Sprint 1 - user login (15 days):

A user/manager can login using their email and password.
Dependencies : initialized Flask framework and connected database.
Tasks:

– initialize Flask framework and database connection
– create a user model, repository and service.
– create a login page fetching a login route.
– test User object creation and persistence with Pytest and API with Postman (body payload, JSON responses, status codes, error messages).

Sprint 2 – Reservations (4 weeks *due to strong dependencies*):
A user can create a reservation and see it on their planning.

Dependencies : Reservation is the core entity of the application. All catalog objects (reservation types, structure types, statuses, audience types, themes) must be implemented (creation, persistence, retrieve) so the front part can fetch the data and display values to select for every types.

Tasks :

- create models for every other object of the system, with repositories.
- create services for the main entities such as "Reservation" and "Structure".
- create API routes ; for catalog entities, focus on a single route with GET method without service.
- create a seed to populate database with catalog entities.
- create a planning page and a reservation creation page.
- test objects creation and persistence with Pytest and API with Postman (body payload, JSON responses, status codes, error messages).

Sprint 3 – Reservation update (15 days):

A user can display details of an existing reservation and update some specific information in it.

Dependencies : previous steps are fully implemented.

Tasks :

- create a PUT route for "Reservation" ; implement manager authorizations.
- re-use the reservation creation screen to display an existing reservation and populate it with the reservation data.
- adapt the JS script so it makes the difference between creation and update situation and fetch a PUT method instead of a POST method.
- display planning pages and reservation differently to manager users so they can access any user reservations.

BUG TRACKING:

Along the sprints, many difficulties have been encountered.

Most of the time bugs were related to the lack of knowledge on the technologies or methods, or syntax errors.

CSS:

- Issues with some extra spaces on some tags (titles, input/label, body) due to browser default values : made some default styles for tags. Created a div with class .field to deal

with input/label displaying.

- No expected behavior of "align-items: center" in a row flexbox : understood it affects the secondary axis and replaced it by "justify-content".
- Wanted a grid calendar, but long paragraphs and padding were causing responsive issues. Finally choose a handier way : flex row display for each day, allowing several reservations for a day. A wrap attribute makes it simpler to solve responsive situations.
- Some elements such as icons were not responding as intended with responsive simulation when using "vh". Solved this by using a "rem" unit or clamp.

SQLAlchemy:

- Many relations with lazy=joined were causing an InvalidRequestError due to multiple rows for a single entity on request. Solved by removing the lazy parameter and letting SQLAlchemy manage separated requests.
- Used the "get_or_404" method in the repository and encountered difficulties with catching the right error type. Finally choose to use session.db.get() so I can check the result and raise the error I want.
- Difficulties with date format. Dates were passed by query parameters in and received as strings, but SQLAlchemy was looking for date objects for requests, raising a type error. Solved this by using the datetime.strptime() method to convert a string into a date object.
- TypeError after some SQLAlchemy requests by a repository because the method .scalars() was forgotten. Rows were sent instead of objects.
- Keyword arguments or positional arguments were encountered while writing requests with filter or filter by. Fixed by using filter when comparisons other than equal comparisons are needed and by finding the right syntax.

Pydantic:

- Tried to use ValueError instead of ValidationError when a field is not validated by Pydantic.
- Pydantic/SQLAlchemy conflict : a UserCreation Pydantic schema was supposed to validate object instantiation and was checking types of ORM models, causing errors : removed UserCreation schema and let SQLAlchemy manage the validation.
- For resource updates, optional fields with value validation were needed. Field() was making them required. Two solutions were used depending on the case : reuse of the

existing payload validator on the PUT method (entire payload expected, non-updatable fields are filtered in the service) or the @field_validator decorator when truly optional fields needed validation with model.dump(exclude_unset=True).

- Forgot to use model.dump() when passing a payload to a validator, leading to an error.
- TypeError after forgetting to deconstruct payload when passed to Pydantic. Solved by using the missing "**payload".

Testing:

- Pytest raises ModuleNotFoundError at launchs, but the app launch successfully otherwise : added the project to sys.path and launched the pytest command from the root of the project.

Flask/JWT/Bcrypt:

- Error when using @jwt_required() without brackets.
- Relationships were not persisted because they were assigned after the session commit. Moved the relationship assignation before the commit.

Python/syntax/algorithms:

- Made a seed script to create an admin without using the service. The password was not hashed, causing an "Invalid salt" error. Solved by hashing the password.
- Error "not JSON serializable" when returning an object by mistake. Solved by using the to_dict() method made for objects or returning lists of to_dict().
- Forgot a return in an exception handler on a route. The route was returning a 200 status code while testing a wrong request to the API. Solved the bug by adding an error message in return and the wanted status code.

JavaScript:

- Bad date incrementation by using "getDay() +1". Fixed it by using "getDate() +1" ; getDay() is used to get the day of the week and getDate() to get the day of the month.
- Difficulties to dynamically render themes with a newline for each theme in a reservation card. Using "
" were displaying the tag itself. Solved the problem by using innerHTML instead of innerText with an array.join("
") as value.
- While displaying audience types, I realized I had not enough data in the model to properly display it whatever the audience types are. I solved this by upgrading my audience types model with a category field and an order_index, allowing to display the

audience types that are class levels in the right order. Since the database had to take those change in account, I rebuilt it and encountered difficulties with the seed : I forgot to change the import after I had copied files in the test directory to have a test seed and production seed.

- While converting dates into strings to pass them in query parameters of my API route, a delay of one day appeared. I was generating the first date as the first day of the month at 00:00 ; while passing it to the "toISOString()" method, timezone was ignored, causing a one hour delay, the first day of the month was now the last day of the previous month at 23:00. Fixed it by using the localeDateString() method with "fr-CA" as parameters so I got the right format for my query parameters.
- I wanted a select input with all structure names available. When a structure is selected, other fields were supposed to filled automatically with the right information, supposing each option tag could store more than the structure name, but also its town, address, phone number and email. Discovered the data-set attribute was the solution.
- I had many try/catch blocks in my file.
  Errors was generating many alerts when the token expires, one for each failed fetch. Solved this by putting as much fetch function as possible in the same "try" block and using "Promise.all([]). If a fetch fails, the first error is caught and the following errors are ignored : only one error alert pops.
  Then, "if" conditions check the response of each fetch before going into rendering.
- At my fifth or sixth addEventListener, I started realizing I could not make a listener for each field of my form. I had a hard time understanding how to use a single one with many "event.target.matches" and what parameters are needed in the matches method. Finally understood I can pass it some CSS syntax.

PRODUCTION ENVIRONMENT:

The application is not deployed and only work on local.

It has been developed on WSL Ubuntu with Python 3.

It uses Flask framework and SQLAlchemy as ORM.

Requirements:

The full list of technologies and there versions can be found in the file "requirements.txt" of the repository.

Here are the main tools used :

- Python – v3.12.3
- SQLite - v3.45.1
- bcrypt - v5.0.0
- Flask - v3.1.2
- Flask-Bcrypt - v1.0.1
- Flask-JWT-Extended - v4.7.1
- flask-restx - v1.3.2
- Flask-SQLAlchemy - v3.1.1
- pydantic - v2.12.5
- pytest - v9.0.2
- SQLAlchemy – v2.0.46

Make sure to install all the dependencies recorded in the "requirements.txt" file before running the app.

Sensitives variables to set up:

Before starting the application, once you have cloned the repository, create a .env file and define the two following variables with strong and secured values :

"SECRET_KEY" – a secret key used by Flask.

"JWT_SECRET_KEY" – a secret key used by JWT to sign your authentication tokens.

Those keys are sensitives and must not be pushed on gitHub.

In the seed_admin.py, a default admin is created. You are strongly encouraged to replace the identifiers (email, password) by your own values.

Database creation and population:

To create your SQLite database file with default catalog types objects, use the seeds provided or adapt it to get your own wished objects.

The following command must be launched once before using the application for the very first time in the "Portfolio_project" working directory :

> python -m seed.seed_db

You might notice a new env.json file is generated in the "exports" directory. Keep it to import it in Postman as variable environment later, or if you need to check the created objects ids.

The SQLite database won't need further configuration.

Running the application:

Launch the back-end application first.

Makes sure the production configuration is used in app/__init__.

You should have line 16:

def create_app(config_class=Config):

And line 5:

from config import Config

When the previous steps are done, use the following command in the root directory "Portfolio_project" :

> python run.py

The following message should appear :

**** Serving Flask app 'app'***

**** Debug mode: off***

WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.

**** Running on http://127.0.0.1:5000***

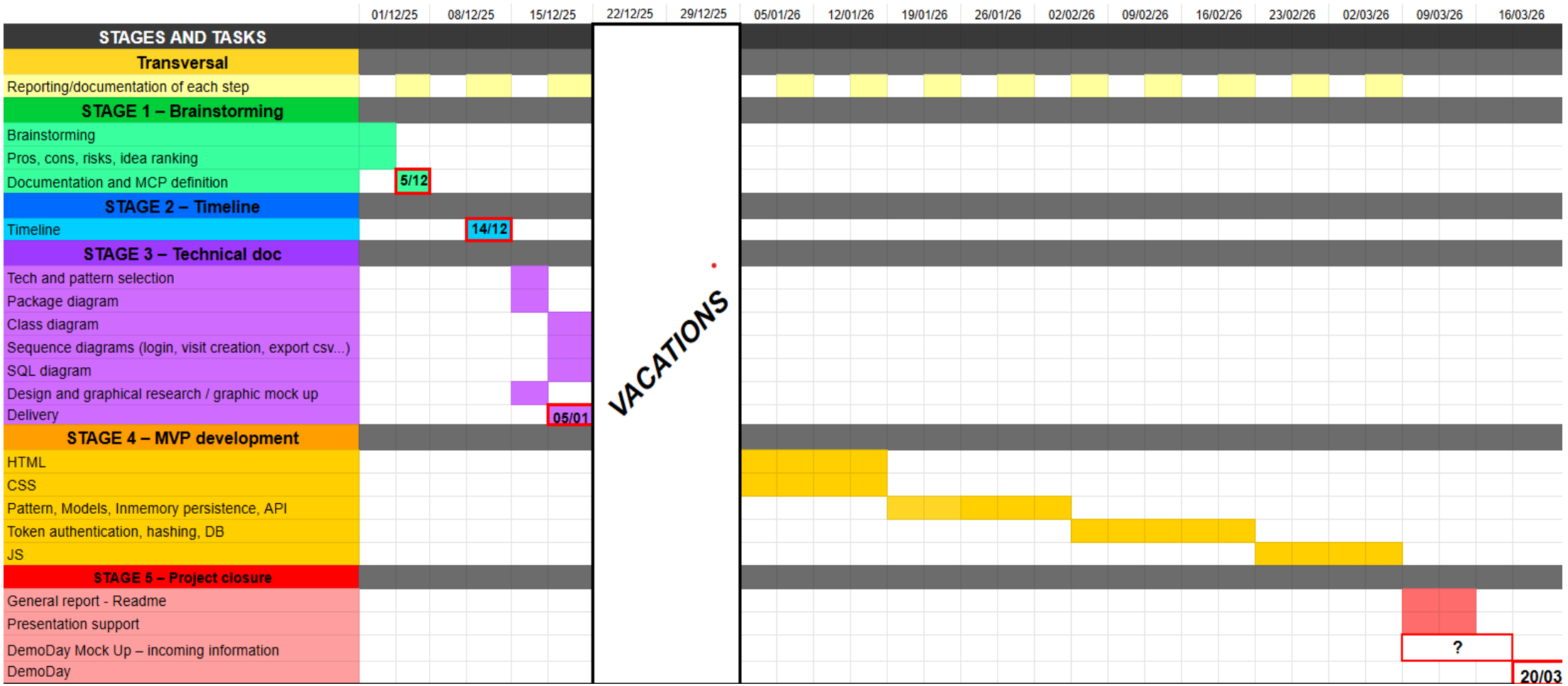
Then, serve the front-end. If you use Vscode, right click on the file "index.html" in front/html.

Otherwise, launch the following command in Portfolio_project/front/html :

> ***python -m http.server 5500***

Your front-end will be accessed by typing the following adress in your browser :

> http://localhost:5500/index.html


01/12/25
08/12/25
15/12/25
22/12/25
29/12/25
05/01/26
12/01/26
19/01/26
26/01/26
02/02/26
09/02/26
16/02/26
23/02/26
02/03/26
09/03/26
16/03/26
STAGES AND TASKS
Transversal
Reporting/documentation of each step
STAGE 1 – Brainstorming
Brainstorming
Pros, cons, risks, idea ranking
Documentation and MCP definition
5/12
STAGE 2 – Timeline
Timeline
14/12
STAGE 3 – Technical doc
Tech and pattern selection
Package diagram
Class diagram
Sequence diagrams (login, visit creation, export csv...)
SQL diagram
Design and graphical research / graphic mock up
Delivery
05/01
VACATIONS
STAGE 4 – MVP development
HTML
CSS
Pattern, Models, Inmemory persistence, API
Token authentication, hashing, DB
JS
STAGE 5 – Project closure
General report - Readme
Presentation support
DemoDay Mock Up – incoming information
?
DemoDay
20/03